

COPY ... FROM

DOUBLE

*

--

LONG

CREATE TABLE

ust N e,

duck,

t Escr=P al=c

* p

B

>= '2

STRING

#

How to Build Data Systems Fall 2026, Lecture 3: From Text to Tree: Parsing SQL

SELECT * FROM ... WHERE

Agenda

Stand-Up (5 min)

Project 1: Defining the Grammar

Exercise 3: From Text to Tree, and Back

- Lexing
- Parsing
- Binding
- Pretty-Printing

5-Minute Stand-Up

In what format is your catalog?

In what format is your data?

What problems did you encounter?

Exercise 2 Recap

Catalog storage

```
public void createTable(String tableName, List<ColumnSpec> columns) { /* ... */ }

public void copyFile(String tableName, String csvFilePath) { /* ... */ }

public List<Object[]> select(String tableName, String columnName,
                             Comparison comparison, Object constant) { /* ... */ }
```

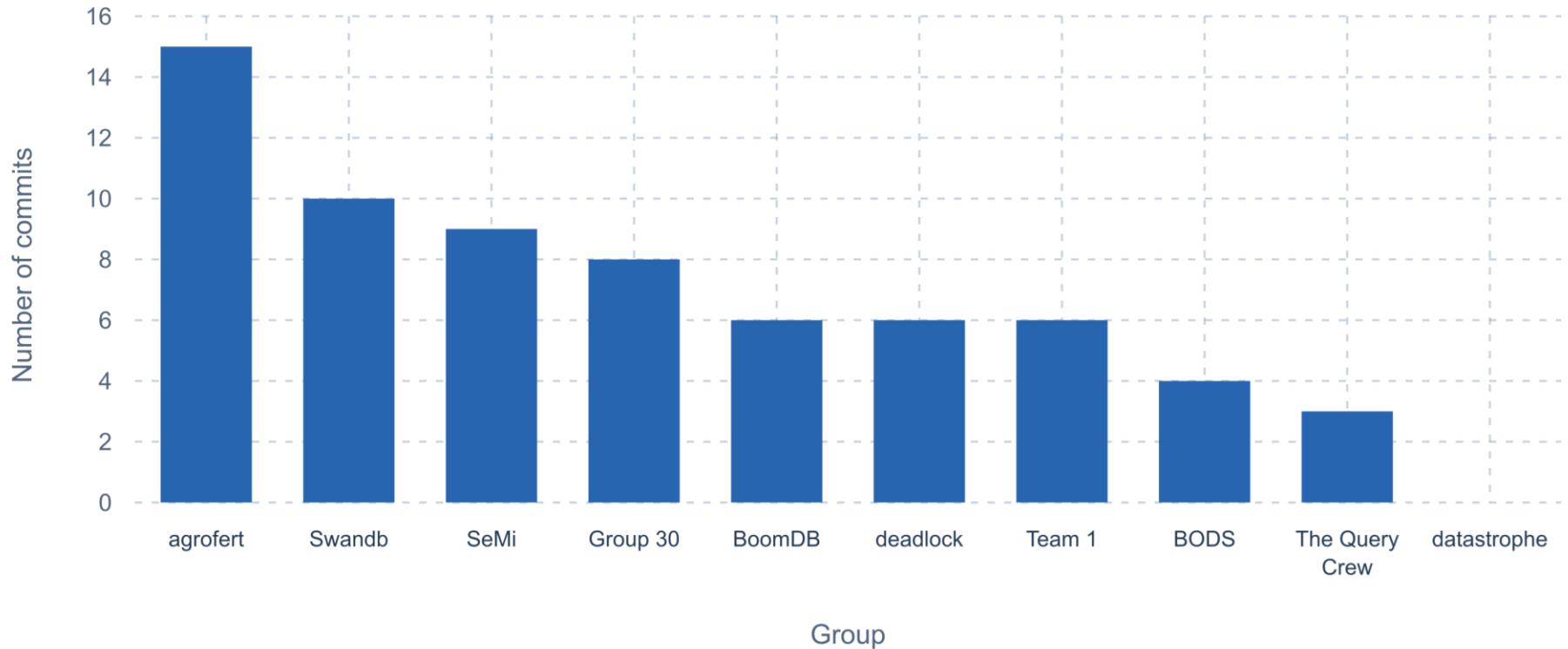
- CSV parsing
- Data storage
- Partition statistics

Partition pruning

Exercise 2 Group Stats

Commits by group

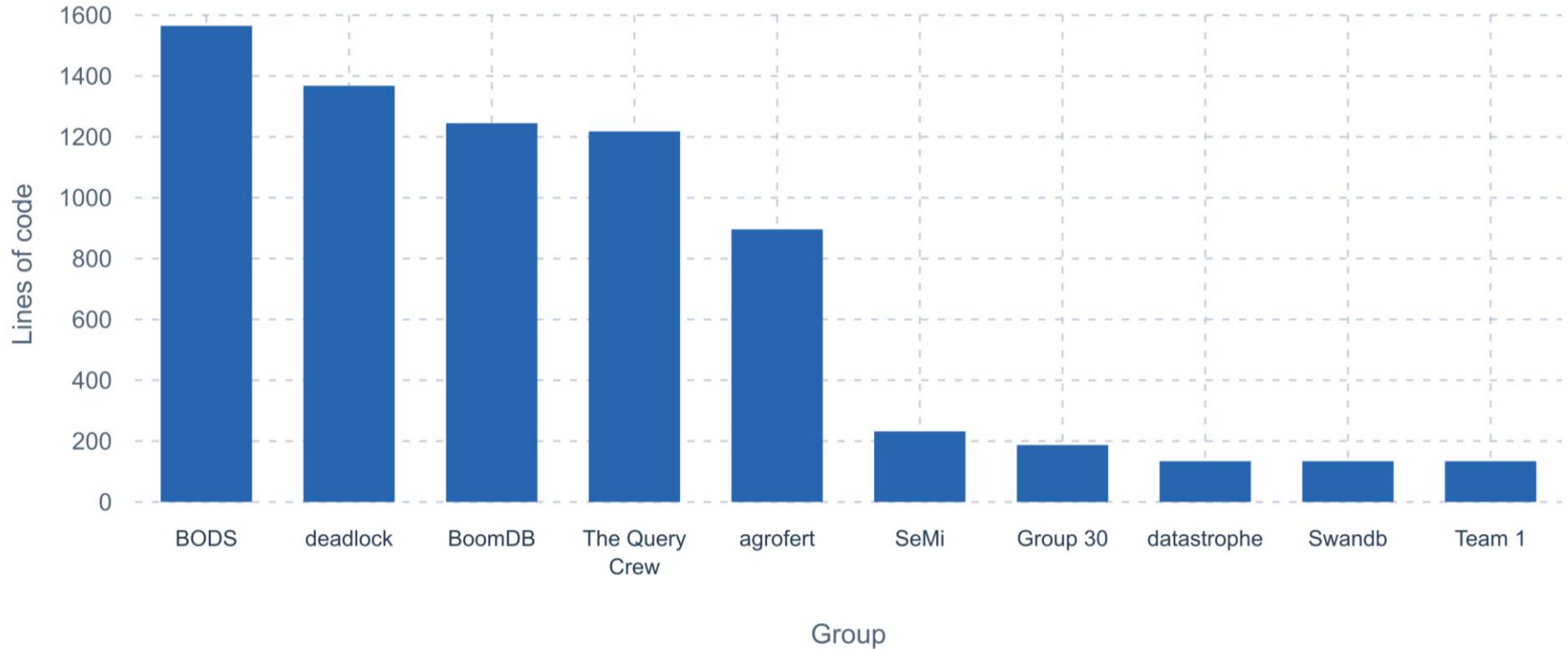
4–9 September 2026



Exercise 2 Group Stats

Lines of code by group

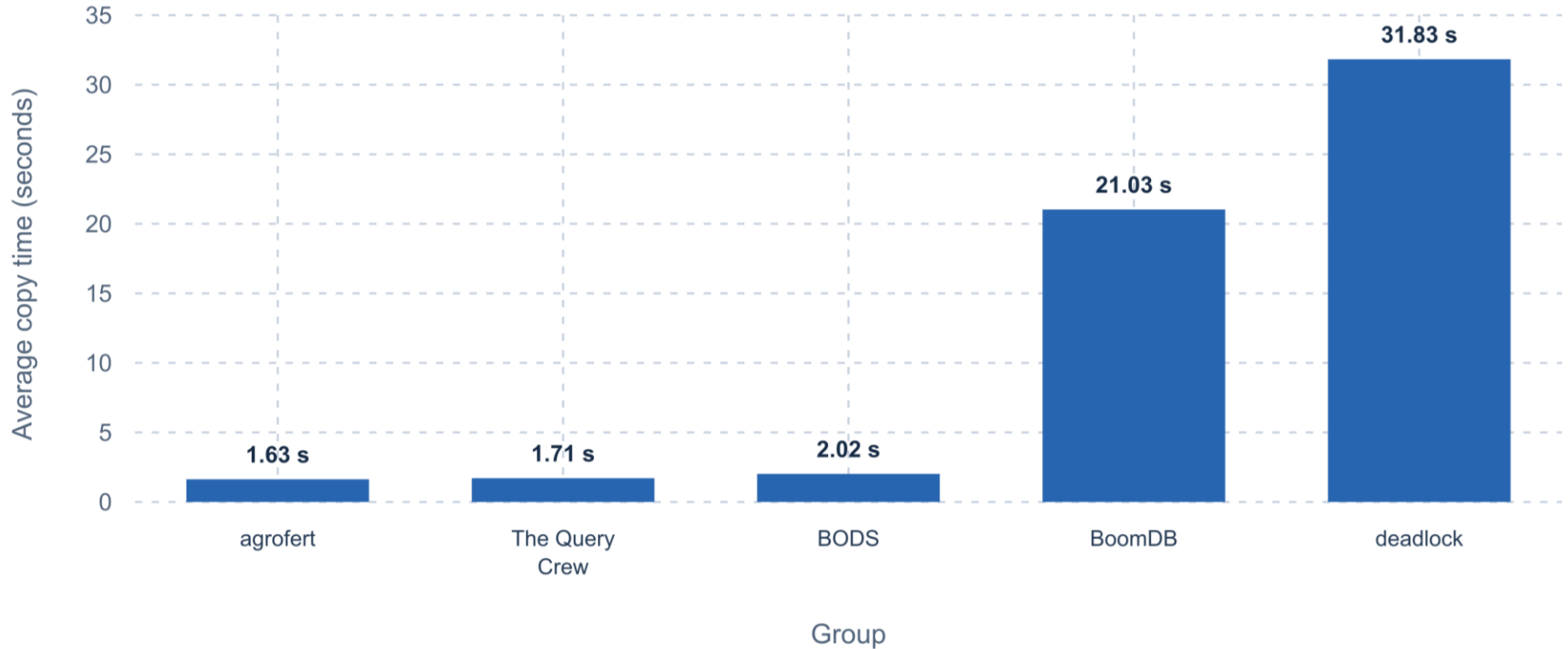
9 September 2026



Exercise 2 Group Stats

Average copy time by group

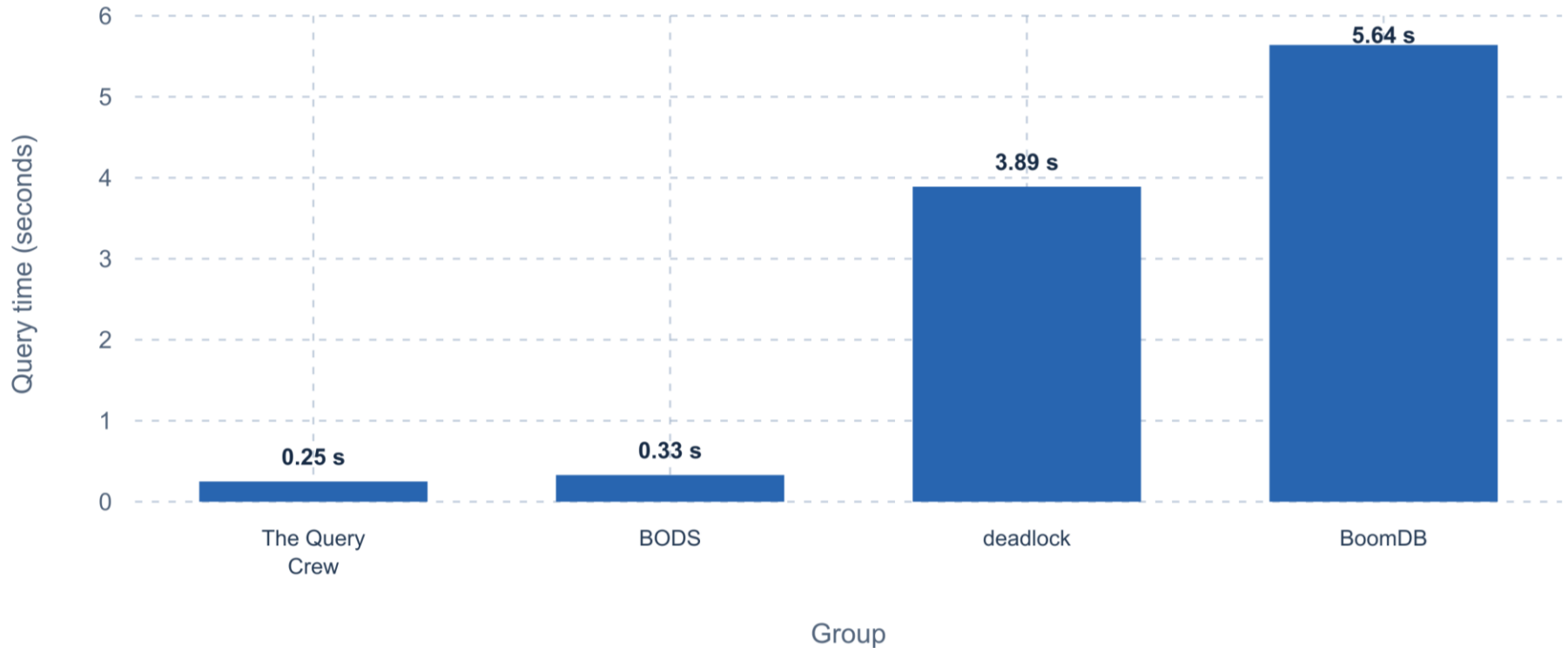
Copying a 100 MB CSV file



Exercise 2 Group Stats

Distance-query time by group

Engine query time for the distance > 900 query



Exercise 3 Preview

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text



```
SqlParser.parse(sql)
```

```
List<Statement> statements
```

AST objects



```
Binder.bind(statements)
```

```
List<BoundStatement> boundStatements
```

AST objects



```
SqlPrinter.print(boundStatements)
```

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text

Project 1 Grammar

```
CREATE TABLE trips (city STRING, distance LONG, price DOUBLE);
COPY trips FROM 'trips.csv';
SELECT * FROM trips WHERE distance > 100;
SELECT * FROM trips;           -- WHERE is optional, as in DuckDB
```

Three statements: CREATE, COPY, SELECT

CREATE

- Three column types: STRING, LONG, DOUBLE

SELECT

- Only SELECT *, no column lists required
- SELECT with optional WHERE clause, no conjunctions yet
- Three comparison operators: = , < , >

Project 1 Grammar

```
CREATE TABLE trips (city STRING, distance LONG, price DOUBLE);  
COPY trips FROM 'trips.csv';  
SELECT * FROM trips WHERE distance > 100;  
SELECT * FROM trips;           -- WHERE is optional, as in DuckDB
```

Special symbols

- Single-quoted strings only
- `--` comment
- `;` terminator

→ Keep it simple and be able to understand the grammar definition. Still works in DuckDB! 

How the Grammar Grows Later

Project 2 adds delete statements

```
DELETE FROM trips WHERE city = 'Aarhus';
```

Project 3 adds table lists and conjunctions (AND)

```
SELECT * FROM orders, customers  
WHERE o_custkey = c_custkey AND ...
```

→ Don't over-build today!

Parsing

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text



```
SqlParser.parse(sql)
```

AST objects

```
List<Statement> statements
```



```
Binder.bind(statements)
```

AST objects

```
List<BoundStatement> boundStatements
```



```
SqlPrinter.print(boundStatements)
```

SQL text

```
SELECT * FROM trips WHERE distance > 100;
```

Did I forget a token?

The lexer groups characters into **tokens**

SELECT Type SELECT Lexeme "SELECT" Line 1 Char 0	* Type STAR Lexeme "*" Line 1 Char 7	FROM Type FROM Lexeme "FROM" Line 1 Char 9	trips Type IDENTIFIER Lexeme "trips" Line 1 Char 14	WHERE Type WHERE Lexeme "WHERE" Line 1 Char 20	distance Type IDENTIFIER Lexeme "distance" Line 1 Char 26	> Type COMPARATOR Lexeme ">" Line 1 Char 35	100 Type LONG_LITERAL Lexeme "100" Line 1 Char 37
---	---	---	--	---	--	---	--

A token is a “rich” substring, including:

- Token type
- Lexeme (the substring)
- Line number
- Char number
- Potentially more information

SQL

```
String sql = "SELECT * FROM trips WHERE distance > 100;"
```

Lexer



How? Split by whitespace, match keywords, special characters, lists of chars.

```
List<Token> = ...
```

```
SELECT STAR FROM IDENTIFIER(trips) WHERE  
IDENTIFIER(distance) COMPARATOR(>)  
LONG_LITERAL(100) ';' 
```

Previous slide forgot
the semicolon token

Lexer

`List<Token> = ...`

```
SELECT STAR FROM IDENTIFIER(trips) WHERE  
IDENTIFIER(distance) COMPARATOR(>  
LONG_LITERAL(100) ';' 
```

Parser

`List<Statement> = ...`

```
SelectStatement  
  String tableName  
  Predicate predicate
```



How? Grammar + Parsing.

Parsing Steps

Step 1

Grammar



Code
Representation
+ Parsing



Iterating the
Parse Tree

ANTLR
parser
tool

Context-Free Grammar

Set of rules of the form:

$$A : a$$


Where

- A is a single nonterminal symbol
- a is a list of nonterminal or terminal symbols and enhanced notation symbols like $| () + * ?$

Grammar Rules

```
breakfast → protein "with" breakfast "on the side" ;  
breakfast → protein ;  
breakfast → bread ;
```

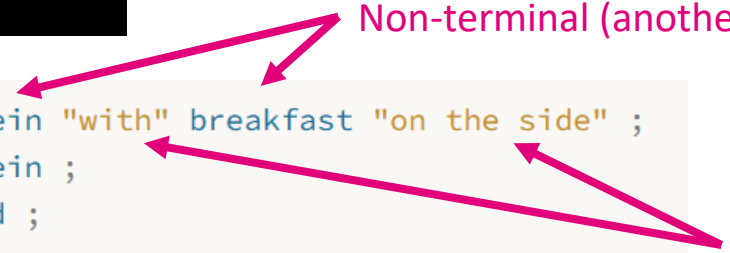
```
protein → crispiness "crispy" "bacon" ;  
protein → "sausage" ;  
protein → cooked "eggs" ;
```

```
crispiness → "really" ;  
crispiness → "really" crispiness ;
```

```
cooked → "scrambled" ;  
cooked → "poached" ;  
cooked → "fried" ;
```

```
bread → "toast" ;  
bread → "biscuits" ;  
bread → "English muffin" ;
```

Non-terminal (another rule)



Terminals (tokens)

Enhanced Notation

Pipe: “pick any”

```
bread → "toast" | "biscuits" | "English muffin" ;
```

Parentheses: grouping

```
protein → ( "scrambled" | "poached" | "fried" ) "eggs" ;
```

*: repeated zero or more

```
crispiness → "really" "really"* ;
```

+: repeated one or more

```
crispiness → "really"+ ;
```

?: appear zero or once

```
breakfast → protein ( "with" breakfast "on the side" )? ;
```

Shortened Grammar

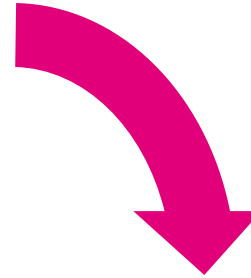
```
breakfast → protein "with" breakfast "on the side" ;  
breakfast → protein ;  
breakfast → bread ;
```

```
protein → crispiness "crispy" "bacon" ;  
protein → "sausage" ;  
protein → cooked "eggs" ;
```

```
crispiness → "really" ;  
crispiness → "really" crispiness ;
```

```
cooked → "scrambled" ;  
cooked → "poached" ;  
cooked → "fried" ;
```

```
bread → "toast" ;  
bread → "biscuits" ;  
bread → "English muffin" ;
```



```
breakfast → protein ( "with" breakfast "on the side" )?  
            | bread ;  
  
protein → "really"+ "crispy" "bacon"  
          | "sausage"  
          | ( "scrambled" | "poached" | "fried" ) "eggs" ;  
  
bread → "toast" | "biscuits" | "English muffin" ;
```

Quiz Time (1)

```
breakfast → protein ( "with" breakfast "on the side" )?  
           | bread ;  
  
protein    → "really"+ "crispy" "bacon"  
           | "sausage"  
           | ( "scrambled" | "poached" | "fried" ) "eggs" ;  
  
bread      → "toast" | "biscuits" | "English muffin" ;
```

Which of the following strings are in the language? (Select all that apply.)

- (a) poached eggs with toast on the side
- (b) crispy bacon
- (c) really really crispy bacon with sausage with biscuits on the side on the side
- (d) toast with sausage on the side
- (e) sausage with toast
- (f) fried eggs with really crispy bacon on the side



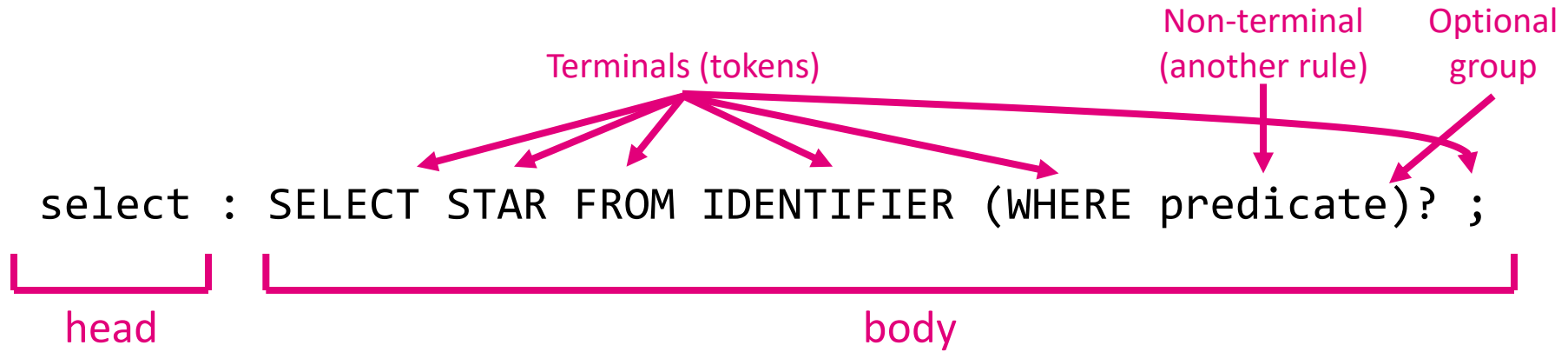
Quiz Time (2)

Which of the following one-line grammars (start symbol s) generate the string **really really really crispy bacon**?

- (a) $s \rightarrow \text{"really"}^* \text{"crispy"} \text{"bacon"} ;$
- (b) $s \rightarrow \text{"really"} \text{"crispy"} \text{"bacon"} ;$
- (c) $s \rightarrow \text{"really"}^+ (\text{"crispy"} \mid \text{"bacon"}) ;$
- (d) $s \rightarrow (\text{"really"} \mid \text{"crispy"} \mid \text{"bacon"})^+ ;$
- (e) $s \rightarrow \text{"really"} s \mid \text{"crispy"} \text{"bacon"} ;$



Context-Free Grammar, SELECT Example



- A grammar is a list of rules; each rule has a head and a body
- Terminals are tokens; nonterminals refer to other rules
- Alternatives with |, repetition with * and +, optional with ?
- [...] denotes a **character set**. It matches exactly one character from the set of characters/ranges you list inside the brackets

Not mentioned in the book. Examples: [123] matches 1, 2, or 3.

[a-z] matches any lower-case character from a to z.

Example ANTLR Grammar Rules for SELECT

grammar SqlSelect;  Name of the grammar

```
select      : SELECT STAR FROM IDENTIFIER (WHERE predicate)? ;
predicate   : IDENTIFIER COMPARATOR literal ;
literal     : STRING_LITERAL | LONG_LITERAL | DOUBLE_LITERAL ;
```

```
SELECT : 'SELECT' ;   FROM : 'FROM' ;   WHERE : 'WHERE' ;
```

```
STAR      : '*' ;
COMPARATOR : '=' | '<' | '>' ;
```

```
IDENTIFIER : [A-Za-z_][A-Za-z_0-9]* ;
LONG_LITERAL : '-'? [0-9]+ ;
DOUBLE_LITERAL : '-'? [0-9]+ '.' [0-9]+ ;
STRING_LITERAL : '\\' ~['\\r\\n']* '\\' ;
```

 ~ = NOT

Only parses keywords in upper-case. Use ANTLR's `caseInsensitive = true` to match any casing.

SQL Specifics: Case-Insensitive Keywords

`select` and `SELECT` are the same keyword

Identifiers keep their case: the lexer preserves `Trips` as written

`Trips` and `trips` are the same table only if your engine says so

- Name resolution is the engine's decision, not the grammar's
- DuckDB matches identifiers case-insensitively; matching exactly accepts less, which would be allowed in your engine

In ANTLR: `options { caseInsensitive = true; }` tells the lexer to match characters and string literals ignoring their case.

Example:

```
options { caseInsensitive = true; }  
// Matches SELECT, select, Select, etc.  
SELECT : 'select';
```

Keywords vs. Identifiers

`SELECT` matches the keyword rule and the identifier rule

Who wins?

1. Lexer rules: the longest match wins ("maximal munch", reading ch. 4):

`SELECTED` is one identifier, not `SELECT` + `ED`

2. On a tie, the rule listed first wins

`SELECT` must not be swallowed by `IDENTIFIER`, which is defined by the order of grammar rules

Literals Defined By Spelling

Spelling	Token	Java type after parsing
12	LONG_LITERAL	Long
12.0	DOUBLE_LITERAL	Double
'12'	STRING_LITERAL	String
-1, -1.5	negative forms	sign is part of the token

Out of scope: escaped quotes inside string literals (' ')*

*SQL has no backslash escapes in an ordinary string literal. A single quote inside a string is written by doubling it, so the name O'Brien is the literal 'O''Brien'.

Comments and Whitespace Are Skipped

```
LINE_COMMENT : '--' ~[\r\n]* -> skip ;  
WS           : [ \t\r\n]+      -> skip ;
```

-> skip: the lexer recognizes comments and whitespace but the parser never sees them

→ The parser cannot preserve a comment, the printer never restores it

Statements, Scripts, EOF

```
script : (statement ';' )+ EOF ;
```

A script is one or more `;`-terminated statements

EOF is what makes trailing garbage an error

```
SELECT * FROM trips; banana
```

^ error because the rule demands EOF here

Without EOF, ANTLR happily stops at the first thing it cannot match

ANTLR (ANother Tool for Language Recognition)

is a parser generator

- you write a grammar (.g4 file) describing a language's syntax as a set of rules
- ANTLR generates Java source code that can parse text written in that language

ANTLR Steps

You write a grammar file (Sql.g4)

- Contains parser rules and lexer rules

ANTLR generates code

- A Lexer class: turns raw text into a stream of tokens
- A Parser class: consumes tokens and builds a **parse tree**, using one ...Context class per rule (e.g. SelectContext)
- A Visitor interface: lets you walk the parse tree and attach your own logic

You write the semantics

- ANTLR only handles recognizing structure
- You implement what happens by extending the base visitor and overriding the methods for the rules you care about

Parsing Steps

Step 2

Grammar



Code
Representation
+ Parsing



Iterating the
Parse Tree

ANTLR
parser
tool

Intermediate Code Representation

ANTLR's intermediate representation is the **parse tree**

- a tree of objects that mirrors the grammar's rule structure
- built by the parser as it consumes tokens
- every rule invocation produces one node in this tree

Building Blocks

For each parser rule, ANTLR generates a `...Context` class.

For example for the following rules:

```
select      : SELECT STAR FROM IDENTIFIER (WHERE predicate)? ;  
predicate   : IDENTIFIER COMPARATOR literal ;  
literal     : STRING_LITERAL | LONG_LITERAL | DOUBLE_LITERAL ;
```

ANTLR produces `SelectContext`, `PredicateContext`, `LiteralContext`. Each a subclass of `ParserRuleContext`, which itself implements `ParseTree`.

Nodes in the Tree

Every node in the tree is one of two kinds:

- **Rule nodes** (`ParserRuleContext` subclasses): internal nodes, one per rule match, e.g. `SelectContext`.
- **Terminal nodes** (`TerminalNode`): leaves, one per matched token, e.g. the actual `SELECT` keyword or an `IDENTIFIER`.

Example Tree

Parsing `SELECT * FROM trips WHERE distance > 100`
builds the following tree:

```
SelectContext
├── TerminalNode: SELECT
├── TerminalNode: STAR
├── TerminalNode: FROM
├── TerminalNode: IDENTIFIER ("trips")
├── TerminalNode: WHERE
└── PredicateContext
    ├── TerminalNode: IDENTIFIER ("distance")
    ├── TerminalNode: COMPARATOR ">"
    └── LiteralContext
        └── TerminalNode: LONG_LITERAL ("100")
```

Accessing Values

Each **Context** class exposes accessor methods matching the rule's grammar elements.

For example **SelectContext** exposes:

```
ctx.IDENTIFIER()    // TerminalNode for the table name
ctx.predicate()     // PredicateContext, or null if the
                    // optional group didn't match
```

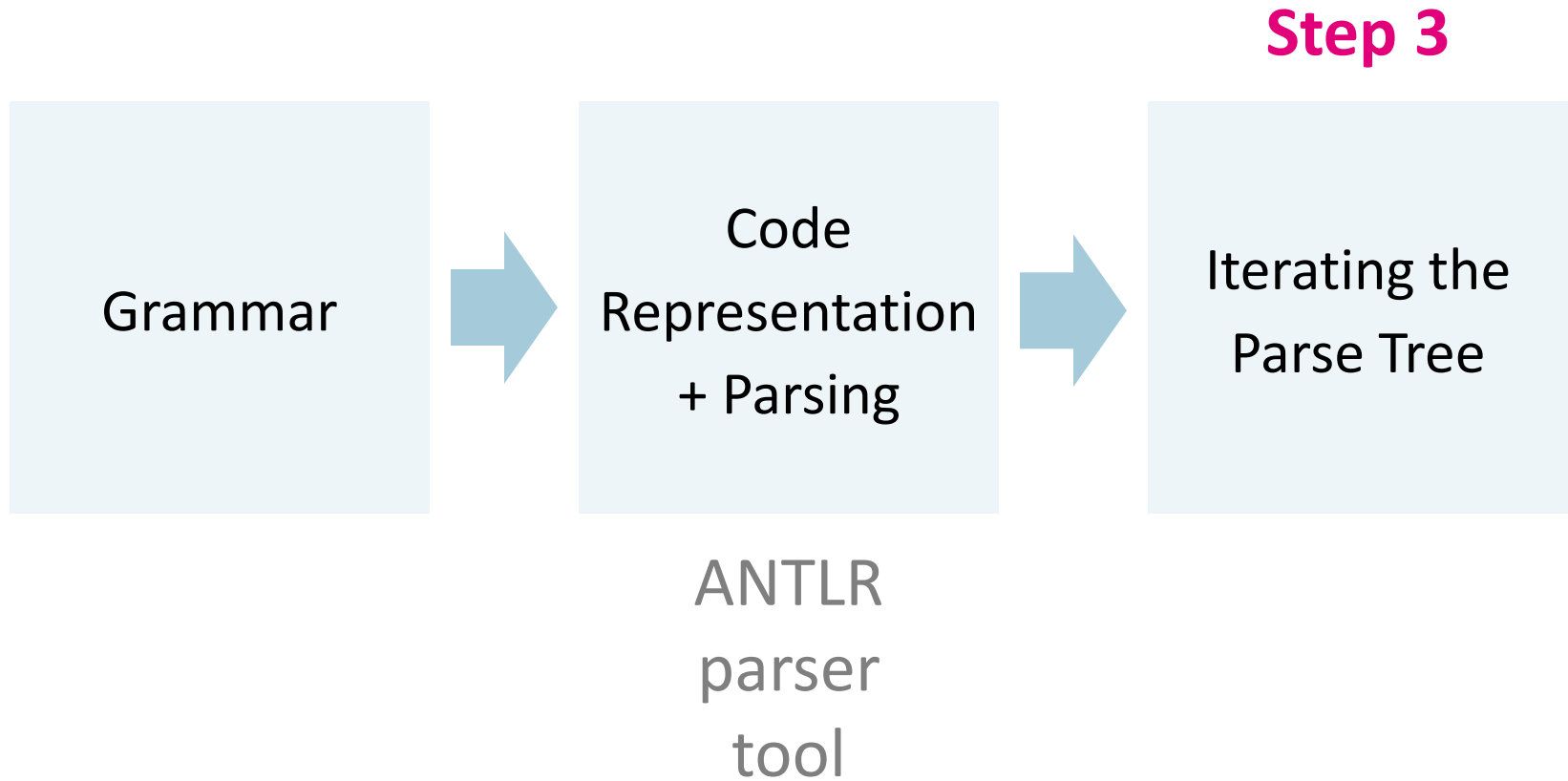
Why “Intermediate” Code Representation

Why it's called "intermediate":

The parse tree sits between raw text and your actual domain objects. It is:

- **Lossless** with respect to grammar structure: every rule match and token is preserved, so it's still tied closely to concrete syntax.
- **Generic:** it doesn't know anything about your semantics (SelectStatement, Predicate); it only knows "a select rule matched, containing these children."

Parsing Steps



Lexer

`List<Token> = ...`

```
SELECT STAR FROM IDENTIFIER(trips) WHERE  
IDENTIFIER(distance) COMPARATOR(>  
LONG_LITERAL(100) ';' 
```

Parser

`List<Statement> = ...`

```
SelectStatement  
  String tableName  
  Predicate predicate
```



How? Grammar + Parsing.

The **visitor** is how you turn the generic parse tree (built by ANTLR) into your own meaningful objects like your `SelectStatement` record.

The problem it solves

- The parse tree ANTLR builds only knows about grammar structure: `SelectContext`, `PredicateContext`, `TerminalNode`.
- It has no idea what a `SelectStatement` or `Predicate` in *your* domain looks like. Something has to walk the tree and translate it. That's the visitor's job.

Visitor Interface

ANTLR generates one `visitX` method per parser rule, all declared on an interface:

“SqlSelect” = Name of the grammar
(see slide 25)



```
public interface SqlSelectVisitor<T> extends ParseTreeVisitor<T>
{
    T visitSelect(SqlSelectParser.SelectContext ctx);
    T visitPredicate(SqlSelectParser.PredicateContext ctx);
    T visitLiteral(SqlSelectParser.LiteralContext ctx);
}
```

`SqlSelectBaseVisitor<T>` is the generated **default implementation** of the `SqlSelectVisitor<T>` interface

Each method defaults to just visiting its children and doing nothing else

```
public class SqlSelectBaseVisitor<T> implements SqlSelectVisitor<T> {  
    public T visitSelect(SelectContext ctx) { return visitChildren(ctx); }  
    public T visitPredicate(PredicateContext ctx) { return visitChildren(ctx); }  
    public T visitLiteral(LiteralContext ctx) { return visitChildren(ctx); }  
}
```

You extend `SqlSelectBaseVisitor<T>` and override only the rule(s) you actually want logic for

Example: Constructing a Record from Parse Tree

```
public record SelectStatement(String tableName, Predicate filters)
implements Statement { }

public class SelectBuilder extends SqlSelectBaseVisitor<Statement> {
    @Override
    public Statement visitSelect(SqlSelectParser.SelectContext ctx) {
        String tableName = ctx.IDENTIFIER().getText();
        Predicate filters = ctx.predicate() != null
            ? buildPredicate(ctx.predicate()) : null;
        return new SelectStatement(tableName, filters);
    }

    private Predicate buildPredicate(SqlSelectParser.PredicateContext ctx) {
        // pull out column name, comparator, literal from ctx
    }
}
```

Example ctd.

```
public class Main {  
    public static void main(String[] args) {  
        String sql = "SELECT * FROM trips WHERE distance > 100;";  
  
        // 1. Lexer: raw text -> tokens  
        SqlSelectLexer lexer = new SqlSelectLexer(CharStreams.fromString(sql));  
  
        // 2. Parser: tokens -> parse tree  
        SqlSelectParser parser = new SqlSelectParser(new CommonTokenStream(lexer));  
        SqlSelectParser.SelectContext tree = parser.select();  
  
        // 3. Visitor: parse tree -> your SelectStatement  
        SelectBuilder builder = new SelectBuilder();  
        SelectStatement stmt = (SelectStatement) tree.accept(builder);  
    }  
}
```

Binding

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text



```
SqlParser.parse(sql)
```

```
List<Statement> statements
```

AST objects



```
Binder.bind(statements)
```

```
List<BoundStatement> boundStatements
```

AST objects



```
SqlPrinter.print(boundStatements)
```

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text

Binding Phase

Parsing only checks *shape*.

```
SELECT * FROM trips WHERE distance > 100;
```

parses fine regardless of

- whether a `trips` table actually exists 
- whether it has a `distance` column 
- whether `100` is of same type as the `distance` column 

The parser has no way to know. Binding is where those errors surface.

The Binder's Checks for SELECT

The table exists

If there is a **WHERE**

- The column exists
- The constant's Java type matches the column type exactly: String ↔ **STRING**, Long ↔ **LONG**, Double ↔ **DOUBLE**

For example: **WHERE** distance = 'x' → binder error

The Binder's Checks for COPY and CREATE TABLE

COPY: the table exists

- Whether the CSV file exists is execution's concern

CREATE TABLE: non-empty column list, no duplicate names

- Whether the table already exists is left to execution, where the check can be made atomically
- For a single-user database (no parallelism), table existence can be checked in the Binder also

Normally: shape problems are the binder's; existence races are execution's. (Execution = Lecture 4)

Example Binding (1)

Switch over sealed classes (compare to Lecture 1)

```
public sealed interface Statement
| | | | permits CreateTableStatement, CopyStatement, SelectStatement { }

public sealed interface BoundStatement
| | | | permits BoundCreateTable, BoundCopy, BoundSelect { }

public BoundStatement bind(Statement statement, Catalog catalog) {
| | | | return switch (statement) {
| | | | | case CreateTableStatement s -> bindCreateTable(s, catalog);
| | | | | case CopyStatement s         -> bindCopy(s, catalog);
| | | | | case SelectStatement s       -> bindSelect(s, catalog);
| | | | };
}
```

Example Binding (2)

```
private BoundStatement bindSelect(SelectStatement s, Catalog catalog) {
    Table table = catalog.lookupTable(s.tableName());           // "trips" -> real table
    List<Column> outputs = table.columns();                      // '*' expanded
    BoundPredicate filter = s.filters() == null
        ? null
        : bindPredicate(s.filters(), table);
    return new BoundSelect(table, outputs, filter);
}

private BoundPredicate bindPredicate(Predicate p, Table table) {
    Column column = table.lookupColumn(p.columnName());         // "distance" -> trips.distance
    requireType(column, p.constant());                          // Long for LONG, no coercion
    return new BoundPredicate(column, p.comparison(), p.constant());
}

/** The constant's Java type must match the column type exactly: String, Long, Double. */
private static void requireType(Column column, Object constant) {
    boolean matches = switch (column.type()) {
        case STRING -> constant instanceof String;
        case LONG   -> constant instanceof Long;
        case DOUBLE  -> constant instanceof Double;
    };
    if (!matches) {
        throw new IllegalArgumentException("type mismatch: " + column.name())
    }
}
```

Pretty Printing

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text



```
SqlParser.parse(sql)
```

```
List<Statement> statements
```

AST objects



```
Binder.bind(statements)
```

```
List<BoundStatement> boundStatements
```

AST objects



```
SqlPrinter.print(boundStatements)
```

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text

Another switch in the PrettyPrinter class

```
public String print(BoundStatement statement) {  
    return switch (statement) {  
        case BoundCreateTable s ->  
            "CREATE TABLE " + s.table().name() + " (" + columnList(s.table().columns()) + ");";  
        case BoundCopy s ->  
            "COPY " + s.table().name() + " FROM " + quote(s.csvFilePath()) + ";";  
        case BoundSelect s ->  
            "SELECT * FROM " + s.table().name() + whereClause(s.filter()) + ";";  
    };  
}  
  
// ... helper methods...
```

Lexing vs. Parsing vs. Binding

Another way to differentiate these is by looking at the errors each stage produces:

- Lexer: unexpected character `$`
- Parser: mismatched input `FORM`, expecting `FROM`
- Binder: unknown table `tripz`

Why Parsing and Binding different steps?

The split keeps both parts simple and testable:

- Parser tests need no catalog
- Binder tests need no grammar details